Web 2 - BINV2150

- **BINV2150-A - Javascript avancé – 80%**
- BINV2150-B - Ergonomie web – 20%

Non-intégré → évaluations séparées

BINV2150-A - Javascript avancé

- 2x2h/semaine → 24 séances
- De la matière à chaque séance, deux fois par semaine
- Déroulé d'une séance :
  - 1. Présentation de la matière (slides) : 30 mins
  - 2. Exercices guidés : 1h30
    - Projet fil rouge
    - Exercices complémentaires
- Enseignants : Tony Leclercq, Jérôme Plumat, Sébastien Strebelle

BINV2150-A - Javascript avancé : matériel

- Slides PDF et code fourni pour les exercices sur moodle
- Slides interactifs sur GitHub (VSCode + slidev)
  - https://github.com/e-vinci/BINV2150A-slides
  - Lisez le README pour savoir comment les utiliser
  - Plus utile pour copier-coller des exemples de code ou interagir avec les slides
- Solutions des exercices sur GitHub (VSCode)
  - https://github.com/e-vinci/BINV2150A-exercices
  - Pas de solutions disponibles sans vos contributions
- Contributions des étudiants
  - Pour les slides et pour les solutions d'exercices
  - Contributions via Pull Request sur GitHub
  - Points bonus en fonction de la qualité des contributions (0.5 pour une solution d'exercice classique)
  - Maximum 4 points bonus pour l'ensemble du cours

BINV2150-A - Javascript avancé : évaluation

- Évaluation : examen écrit uniquement
  - Conception d'un site web complet (front + back) en 2h
  - Évaluation fonctionnelle uniquement, pas de notation sur le code
  - Examen blanc à la fin du cours pour se préparer
- Mêmes modalités en première et en deuxième session
- Note de l'acap : examen écrit + bonus pour contributions
- Note de l'UE : 80% pour BINV2150-A, 20% pour BINV2150-B

Plan du cours

- **Partie 1 — Backend (S01-S04)** : extensions TypeScript, API REST et documentation, JWT, hachage et promesses/async
- **Partie 2 — Frontend "old school" (S05-S06)** : DOM, querySelector, événements
- **Partie 3 — React (S07-S20)** : composants, props, état, routage, context, useEffect, architecture MVVM
- **Partie 4 — Liaison front-back (S21-S24)** : fetch, CORS/proxy, session, requêtes authentifiées
- **Conclusion** : examen blanc

Projet fil rouge : MiamMiam — Application web (front+back) de recettes de cuisine

Web 2 – Séance 01

Extensions TypeScript

- Programmation fonctionnelle
- Optional Chaining `?.`
- Nullish Coalescing `??`
- Non-null Assertion `!`
- Spread Operator `...`
- Destructuring
- Type Guard

Programmation fonctionnelle

Paradigme de programmation qui privilégie les fonctions pures, l'immuabilité et l'absence d'effets de bord.

- Variables immuables (const) plutôt que mutables (let)
- Fonctions pures : mêmes entrées → mêmes sorties, pas d'effets de bord
- Fonctions d'ordre supérieur : fonctions qui prennent des fonctions en argument ou retournent des fonctions
- Composition de fonctions : combiner des fonctions pour créer de nouvelles fonctions

```
const numbers = [1, 2, 3, 4, 5];
// Fonction pure : ne modifie pas le tableau original
const doubled = numbers.map(n ⇒ n * 2); // [2, 4, 6, 8, 10]
const even = numbers.filter(n ⇒ n % 2 === 0); // [2, 4]
// Composition de fonctions : combiner map et filter
const doubledEven = numbers.filter(n ⇒ n % 2 === 0).map(n ⇒ n * 2); // [4, 8]
// Fonction d'ordre supérieur : prend une fonction en argument
function applyToAll(arr: number[], fn: (n: number) ⇒ number): number[] {
  return arr.map(fn);
}
const squared = applyToAll(numbers, n ⇒ n * n); // [1, 4, 9, 16, 25]
```

Optional Chaining `?.`

Accéder à une propriété qui peut ne pas exister, en toute sécurité.

```ts
interface Recipe {
  title: string;
  steps?: { description: string; duration: number }[];
  send: () => void;
}

const recipe: Recipe | undefined = getRecipe();

// ❌ Avant : erreur possible
const prepTime = recipe.steps[0].duration; // runtime error si steps est undefined

// ✅ Après : avec optional chaining
const prepTime = recipe?.steps?.[0]?.duration; // retourne undefined si une étape est manquante, ou la valeur si elle ex

// ✅ Après : optional chaining avec fonction
recipe?.send(); // appelle send() si recipe existe, sinon ne fait rien
```



Nullish Coalescing `??`

Fournir une valeur par défaut si la valeur est null ou undefined.

```
interface Recipe {
  title: string;
  servings?: number;
}

function getRecipe(): Recipe | undefined {
   ...
}

// ❌ Avant : peut retourner undefined
const recipe: Recipe | undefined = getRecipe();

// ✅ Après : valeur par défaut si la fonction retourne undefined
const finalRecipe: Recipe = recipe ?? { title: "Recette par défaut", servings: 4 };

// ✅ Après : valeur par défaut si servings est undefined
const servings: number = recipe?.servings ?? 4; // retourne 4 seulement si servings ou recipe est undefined
```



Non-null Assertion `!`

Affirmer au compilateur TypeScript qu'une valeur n'est PAS null/undefined.

```ts
interface Recipe {
  title: string;
  author?: string;
}

const recipe1: Recipe = { title: "Cookies" };
const recipe2: Recipe = { title: "Pancakes", author: "Alice" };

// ❌ Erreur TypeScript : author peut être undefined
const name1 = recipe1.author.toUpperCase();

// ✅ Assertion : "je sais que c'est défini"
const name2 = recipe2.author!.toUpperCase();

// À utiliser avec prudence ! Vérifiez vraiment.
```



Non-null Assertion `!`

Autre exemple.

```
function useRecipe(recipe: Recipe) {
  ...
}

const recipe: Recipe | undefined = getRecipe();

// ❌ Erreur TypeScript : recipe peut être undefined
useRecipe(recipe);

// ✅ Assertion : "je sais que c'est défini"
useRecipe(recipe!);
```



Spread Operator : Objets

Copier et fusionner des objets facilement.

```
interface Recipe {
  title: string;
  servings: number;
  tags?: string[];
}

const recipe1: Recipe = { title: "Cookies", servings: 12 };
const recipe2: Recipe = { title: "Brownies", servings: 8, tags: ["dessert"] };

// Copie
const copy = { ...recipe1 };
copy.title = "Muffins";
console.log(copy); // { title: "Muffins", servings: 12 } - modifié
console.log(recipe1); // { title: "Cookies", servings: 12 } — inchangé

// ATTENTION : les objets imbriqués ne sont pas copiés en profondeur (shallow copy)
const copyWithTags = { ...recipe2 };
copyWithTags.tags?.push("chocolate");
console.log(copyWithTags); // { title: "Brownies", servings: 8, tags: ["dessert", "chocolate"] } - modifié
console.log(recipe2); // { title: "Brownies", servings: 8, tags: ["dessert", "chocolate"] } - aussi modifié
```



Spread Operator : Objets

Copier et fusionner des objets facilement.

```
const obj1 = { a: 1, b: 2 };
const obj2 = { b: 3, c: 4 };

// Fusion avec override
const merged = { ...obj1, ...obj2 };
console.log(merged); // { a: 1, b: 3, c: 4 } - obj2 override obj1

// Ajout de propriétés
const withD = { ...obj1, d: 4 };
console.log(withD); // { a: 1, b: 2, d: 4 } - ajout de la propriété d à obj1
```



Spread Operator : Tableaux

Combiner et dupliquer des tableaux.

```
const ingredients1 = ["farine", "sucre"];
const ingredients2 = ["oeufs", "beurre"];

// Fusion
const allIngredients = [...ingredients1, ...ingredients2]; // ["farine", "sucre", "oeufs", "beurre"]

// Ajout d'éléments
const withSalt = [...ingredients1, "sel"]; // ["farine", "sucre", "sel"]

// Copie
const copy = [...ingredients1]; // ["farine", "sucre"]
copy[0] = "riz";
console.log(copy); // ["riz", "sucre"] - modifié
console.log(ingredients1); // ["farine", "sucre"] — inchangé
```



Destructuring : Objets

Extraire les propriétés d'un objet en variables.

```
interface Ingredient {
  name: string;
  quantity: number;
  unit: string;
  temperature?: number;
}

const ingredient: Ingredient = { name: "farine", quantity: 200, unit: "g" };

// Destructuring simple
const { name, quantity } = ingredient;

// Renommage
const { name: ingredientName, unit: mesure } = ingredient;

// Valeur par défaut (si la propriété est absente)
const { temperature = 180 } = ingredient;
```



Destructuring : Paramètres de fonction

Extraire les propriétés d'un objet directement dans les paramètres d'une fonction.

```
// ❌ Avant : sans destructuring
function printIngredient(ingredient: Ingredient) {
  const name = ingredient.name;
  const quantity = ingredient.quantity;
  const unit = ingredient.unit;
  console.log(`${quantity}${unit} de ${name}`);
}

// ✅ Après : avec destructuring
function printIngredient({ name, quantity, unit }: Ingredient) {
  console.log(`${quantity}${unit} de ${name}`);
}
```



Type guard

Vérifier le type d'une variable pour affiner le type dans un bloc de code.

```
interface Recipe {
  title: string;
  ingredients: string[];
}

function isRecipe(obj: any): obj is Recipe {
  return obj &&
      typeof obj.title === "string" &&
      Array.isArray(obj.ingredients) &&
      obj.ingredients.every(str => typeof str === "string");
} // Vérifier entièrement et uniquement les propriétés de l'objet

function process(obj: any) {
  if (isRecipe(obj)) {
    // TypeScript sait que obj est de type Recipe ici
    console.log(obj.title);
  } else {
    console.log("Ce n'est pas une recette valide.");
  }
}
```



Type guard

Autre exemple.

```
interface Ingredient {
  name: string;
  quantity: number;
}

interface Recipe {
  title: string;
  servings: number;
  ingredients: Ingredient[];
}

function isIngredient(obj: any): obj is Ingredient {
  return obj && typeof obj.name === "string" && typeof obj.quantity === "number";
}

function isRecipe(obj: any): obj is Recipe {
  return obj &&
      typeof obj.title === "string" &&
      typeof obj.servings === "number" &&
      Array.isArray(obj.ingredients) &&
      obj.ingredients.every(isIngredient);
}
```

Type guard

Encore un exemple.

```
interface Recipe {
  title: string;
  ingredients: Ingredient[];
  steps?: string[];
}

function isRecipe(obj: any): obj is Recipe {
  if (!obj) return false;
  if (typeof obj.title !== "string") return false;
  if (!Array.isArray(obj.ingredients)) return false;
  for (const ingredient of obj.ingredients) {
    if (!isIngredient(ingredient)) return false;
  }
  if (obj.steps) {
    if (!Array.isArray(obj.steps)) return false;
    for (const step of obj.steps) {
      if (typeof step !== "string") return false;
    }
  }
  return true;
}
```



Récapitulatif Séance 01

- **Programmation fonctionnelle** – Fonctions pures, immuabilité, fonctions d'ordre supérieur, composition
- **Optional Chaining** `?.` – Accès sûr aux propriétés optionnelles
- **Nullish Coalescing** `??` – Valeur par défaut si null/undefined
- **Non-null Assertion** `!` – Affirmer qu'une valeur n'est pas null/undefined
- **Spread Operator** `...` – Copier et fusionner objets/tableaux
- **Destructuring** – Extraire des valeurs clairement
- **Type Guard** – Vérifier le type d'une variable

Prochaine séance : Séance 02 – Rappels Requêtes HTTP, Conventions REST, Documentation d'API

Exercices

Structure attendue

- BINV2150-A
  - slides
    - seance01.pdf
    - seance02.pdf
    - …
  - exercices
    - MiamMiam
      - **backend**
      - frontend
    - exercices-complementaires
      - EC01
      - EC02
      - …

Exercice filé S01

1. Téléchargez le boilerplate du backend du projet fil rouge MiamMiam, décompressez-le et ouvrez-le dans VSCode
2. Analysez le code existant et retrouvez les différentes fonctionnalités déjà implémentées
3. Trouvez les endroits où vous pouvez appliquer les notions vues dans cette séance (optional chaining, spread operator, destructuring, type guard)
4. Appliquez ces notions dans le code existant
5. Testez votre code pour vous assurer qu'il fonctionne correctement